

Task 03 — Spring Boot Configuration & Environment Profiles

Step 1 — Understand Configuration

application.properties

This is a configuration file used by Spring Boot.

Location: src/main/resources/application.properties

Example:

```
spring.application.name=user-service
```

```
server.port=8080
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/userdb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=root
```

application.yml

Spring Boot also supports YAML configuration.

Instead of:

```
spring.application.name=user-service
```

```
server.port=8080
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/userdb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=root
```

you can write: YAML

```
spring:
```

```
  application:
```

```
    name: user-service
```

```
datasource:
```

```
  url: jdbc:mysql://localhost:3306/userdb
```

username: root

password: root

server:

port: 8080

Use YAML:

YAML is easier to read when configuration becomes large.

For example:

spring:

datasource:

url: ...

username: ...

password: ...

clearly shows that all three properties belong to spring.datasource.

Important

Usually, choose **either**:

application.properties

or:

application.yml

for the same configuration. For your task, I recommend using **YAML**, because the next steps involve profiles

Environment Variables

Environment variables allow us to keep configuration outside our source code.

don't put a production password directly in Git:

spring:

datasource:

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

Spring Boot reads:

DB_USERNAME

DB_PASSWORD

Profiles

This is one of the most important concepts in your JIRA task.

Different environments need different configurations.

Example:

Development



userdb_dev

Testing



userdb_test

Production



userdb_prod

Instead of changing your configuration manually every time, Spring Boot provides **Profiles**.

Create:

src/main/resources/

├— application.yml

├— application-dev.yml

├— application-test.yml

└— application-prod.yml

application.yml

Put common configuration here:

spring:

 application:

 name: user-service

This applies to all environments.

application-dev.yml

Development configuration:

server:

 port: 8081

spring:

 datasource:

 url: jdbc:mysql://localhost:3306/userdb_dev

 username: root

 password: root

jpa:

 hibernate:

 ddl-auto: update

 show-sql: true

application-test.yml

Testing configuration:

server:

 port: 8082

spring:

 datasource:

 url: jdbc:mysql://localhost:3306/userdb_test

username: root

password: root

jpa:

hibernate:

ddl-auto: create-drop

show-sql: false

application-prod.yml

Production configuration:

server:

port: 8080

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_prod

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

jpa:

hibernate:

ddl-auto: validate

show-sql: false

Configuration Classes

A configuration class is a Java class where you define application configuration or Spring beans.

Example:

```
package com.example.userservice.config;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class AppConfig {  
}
```

@Configuration tells Spring:

This class contains configuration for the application.

Example with @Bean

```
package com.example.userservice.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
  
@Configuration  
public class AppConfig {  
  
    @Bean  
  
    public String applicationName() {  
        return "User Service";  
    }  
}
```

Spring creates and manages this object as a Bean.

Step 2 — Convert configuration to YAML

You should create it here:

src/main/resources/application.yml

If you currently have:

application.properties

you can convert the configuration into YAML.

Application name

Properties

spring.application.name=user-service

YAML

spring:

application:

name: user-service

Port

DEV → 8081

TEST → 8082

PROD → 8083

We will configure these in separate profile files.

Database configuration

Example, using MySQL:

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb

username: root

password: root

This tells Spring Boot:

Database type → MySQL

Database → userdb

Username → root

Password → root

JPA configuration

Add:

spring:

jpa:

hibernate:

ddl-auto: update

show-sql: true

Meaning

ddl-auto: update

Spring/Hibernate will update the database schema based on your entities.

show-sql: true

Hibernate SQL queries will appear in the console.

Logging configuration

You can configure logging like this:

logging:

level:

root: INFO

com.example.userservice: DEBUG

Meaning:

root → INFO

your application package → DEBUG

Replace:

com.example.userservice

with your actual Java package.

Complete application.yml

spring:

application:

name: user-service

jpa:

hibernate:

ddl-auto: update

show-sql: true

logging:

level:

root: INFO

com.example.userservice: DEBUG

currently have application.properties

Suppose your existing file contains:

properties

spring.application.name=user-service

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/userdb

spring.datasource.username=root

spring.datasource.password=root

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

logging.level.root=INFO

logging.level.com.example.userservice=DEBUG

Convert it to: YAML

spring:

application:

name: user-service

datasource:

url: jdbc:mysql://localhost:3306/userdb

username: root

password: rootdriver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: update

show-sql: true

server:

port: 8080

logging:

level:

root: INFO

com.example.userservice: DEBUG

The configuration is the same; only the format has changed.

Step 3 — Create Profiles

In:

src/main/resources/

Your project should be:

src

└─ main

└─ resources

└─ application.yml

└─ application-dev.yml

└─ application-test.yml

└─ application-prod.yml

Step 4 — Configure Different Ports

Development

Open: application-dev.yml

Add:

server:

port: 8081

we can also configure the development database:

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_dev

username: root

password: root

jpa:

hibernate:

ddl-auto: update

show-sql: true

So DEV means:

Application

↓

Port 8081

↓

userdb_dev

Test

Open: application-test.yml

Add:

server:

port: 8082

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_test

username: root

password: root

jpa:

hibernate:

ddl-auto: create-drop

show-sql: false

So TEST means:

Application

↓

Port 8082

↓

userdb_test

Using a separate test database is important so your testing doesn't accidentally modify development data.

Production

Open: application-prod.yml

Add:

server:

port: 8083

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_prod

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

jpa:

hibernate:

ddl-auto: validate

show-sql: false

So PROD means:

Application



Port 8083



userdb_prod

Notice:

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

We don't want to put real production credentials directly into the code.

Step 5 — Configure different databases

Use separate database configurations for each environment.

DEV

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_dev

TEST

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_test

PROD

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_prod

Therefore:

DEV

↓

userdb_dev

TEST

↓

userdb_test

PROD

↓

userdb_prod

Complete files

application.yml

This contains common configuration.

spring:

application:

name: user-service

logging:

level:

root: INFO

application-dev.yml

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_dev

username: root

password: root

driver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: update

show-sql: true

logging:

level:

root: INFO

com.example.userservice: DEBUG

application-test.yml

server:

port: 8082

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_test

username: root

password: root

driver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: create-drop

show-sql: false

logging:

level:

root: INFO

application-prod.yml

server:

port: 8083

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_prod

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

driver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: validate

show-sql: false

logging:

level:

root: WARN

Replace:

com.example.userservice

with your actual package name.

Step 6 — Activate Profiles

Now we have three configurations, but Spring Boot needs to know **which one to use**.

We can also activate profiles from the terminal.

DEV

```
mvn spring-boot:run -Dspring-boot.run.profiles=dev
```

TEST

```
mvn spring-boot:run -Dspring-boot.run.profiles=test
```

PROD

```
mvn spring-boot:run -Dspring-boot.run.profiles=prod
```

Step 7 — Environment variables

spring:

datasource:

url: \${DB_URL}

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

Spring Boot will get the actual values from the environment.

Update application-prod.yml

Go to:

src/main/resources/application-prod.yml

Use:

server:

port: 8083

spring:

datasource:

url: \${DB_URL}

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

driver-class-name: com.mysql.cj.jdbc.Driver

jpa:

hibernate:

ddl-auto: validate

show-sql: false

logging:

level:

root: WARN

Now your code doesn't contain the actual username/password.

Set Environment Variables

IntelliJ IDEA

Go to:

Run



Edit Configurations



Your Spring Boot Application



Environment variables

Add:

DB_URL=jdbc:mysql://localhost:3306/userdb_prod

DB_USERNAME=root

DB_PASSWORD=root123

Also select the profile: prod

Then run the application.

Windows Command Prompt

You can also set them temporarily in CMD:

```
set DB_URL=jdbc:mysql://localhost:3306/userdb_prod
```

```
set DB_USERNAME=root
```

```
set DB_PASSWORD=root123
```

Then: `mvn spring-boot:run -Dspring-boot.run.profiles=prod`

Spring Boot reads:

DB_URL

DB_USERNAME

DB_PASSWORD

automatically.

Step 8 — Never Hard-Code Secrets

Check the source code and remove passwords, API keys, tokens and database credentials.

Use environment variables:

spring:

datasource:

username: \${DB_USERNAME}

password: \${DB_PASSWORD}

url: \${DB_URL}

For an API key:

external:

api-key: \${API_KEY}

For a token/secret:

jwt:

secret: \${JWT_SECRET}

The actual values stay outside your source code.

What can check:

Search your project for things like:

password

api-key

token

secret

username

jdbc URL

Check files such as:

application.yml

application-dev.yml

application-test.yml

application-prod.yml

For example, change: password: root123

to:

password: \${DB_PASSWORD}

Step 9 — Test Configurations**Application starts**

Run your Spring Boot application and make sure there are no startup errors.

For example:

Started UserServiceApplication

Tomcat started on port 8081

That means the application started successfully.

Correct database is selected

Each profile should use its own database.

DEV → userdb_dev

TEST → userdb_test

PROD → userdb_prod

Example, your application-dev.yml:

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_dev

Your application-test.yml:

spring:

datasource:

url: jdbc:mysql://localhost:3306/userdb_test

Your application-prod.yml:

spring:

datasource:

url: \${DB_URL}

Correct port is used

Your task specifies:

DEV → 8081

TEST → 8082

PROD → 8083

So:

DEV

server:

port: 8081

TEST

server: port: 8082

PROD

server:

port: 8083

Correct profile is active

When you run DEV: `mvn spring-boot:run -Dspring-boot.run.profiles=dev`

The console should indicate that dev is active.

For TEST: `mvn spring-boot:run -Dspring-boot.run.profiles=test`

For PROD: `mvn spring-boot:run -Dspring-boot.run.profiles=prod`

Step 9 — Final verification

Profile	Expected Port	Expected Database
DEV	8081	userdb_dev
TEST	8082	userdb_test
PROD	8083	userdb_prod